Unleash: Automated Pareto-Optimal Warning Reduction for Rust via Span-Based Agentic Refactoring

Anonymous Author(s)

Abstract

Compiler warnings are a persistent indicator of code quality debt in Rust projects. Despite Clippy's rich diagnostic suite, empirical surveys show that real-world Rust crates on `crates.io` average 21 warnings per thousand lines of code (KLOC), with a highly skewed Pareto distribution where the top-5 lint types account for over 80% of all warnings. Reducing warnings manually is tedious and error-prone, while naive automated tools (`cargo clippy -fix`) handle only a small fraction of fixable lints. We present Unleash, a framework for automated Pareto-optimal warning reduction in Rust. Unleash combines three components: (1) rust-diagnostics, a CLI utility that embeds Clippy diagnostics as inline source annotations and produces a structured JSON cache of all warning spans; (2) a span-based Python refactoring engine that applies precise textual replacements from the diagnostic cache to eliminate entire lint classes in one pass; and (3) an agentic loop powered by a large language model (LLM) that iteratively targets the highest-frequency lint type, applies the appropriate fix strategy, verifies compilation, and recounts until the warning density falls below the 18/KLOC target. We evaluate Unleash on openhitls-rs, a 232K-LOC Rust port of the OpenHiTLS cryptographic library containing 8,298 warnings at 35.69/KLOC across 26 lint types. Unleash reduces warnings from 35.69 to 14.72/KLOC (4,923 warnings eliminated, 59% reduction) across five automated fix passes: machine-applicable auto-fix (−1,930), span-based type suffix annotation (−982), span-based `# Errors` doc insertion (−655), and span-based `try_from` cast replacement (−593), with all intermediate builds passing. Unleash reaches the 18/KLOC threshold in under 35 minutes of elapsed agentic time, compared to an estimated 80+ person-hours for equivalent manual remediation. To our knowledge, this is the first framework combining compiler diagnostic spans, structured JSON caching, and LLM-driven iterative refactoring to achieve Pareto-optimal warning reduction at scale.

CCS Concepts

• **Software and its engineering** → **Language features**; **Software defect analysis**; *Automatic programming*.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ASE '26, Munich, Germany
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2026/10
https://doi.org/XXXXXXX.XXXXXXX

Keywords

Rust, Clippy, static analysis, warning reduction, automated refactoring, large language models, Pareto distribution, span-based rewriting, code quality

ACM Reference Format:
Anonymous Author(s). 2026. Unleash: Automated Pareto-Optimal Warning Reduction for Rust via Span-Based Agentic Refactoring. In *Proceedings of Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering (ASE '26).* ACM, New York, NY, USA, 6 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Rust's type system and borrow checker provide strong safety guarantees, but the language's rich lint suite (Clippy) generates a large volume of *warnings*: diagnostics that do not prevent compilation but indicate code quality issues, unsafe patterns, or deviations from idiomatic Rust. An empirical study by Li *et al.* [6] of 2,669 open-source Rust crates found that the average crate carries 21 warnings per thousand lines of code (KLOC), with a highly skewed Pareto distribution: the top-5 lint types account for over 80% of all warnings. The same study demonstrated that a density below 18/KLOC—achievable by eliminating only the top few lint types—places a project below the crates-io average baseline.

Despite the availability of `cargo clippy -fix`, which applies *machine-applicable* suggestions automatically, a substantial fraction of warnings require non-trivial semantic rewrites (*e.g.* replacing arithmetic operators with wrapping variants, inserting `try_from` conversions, annotating function signatures with error documentation). Eliminating these warnings manually across large codebases is both time-consuming and inconsistent.

We address this problem with Unleash (**U**nified **N**on-trivial **L**int **E**limination via **A**gentic **S**pan-based **H**andling), a framework that automates Pareto-optimal warning reduction via three integrated components:

(1) **rust-diagnostics**: a CLI utility that invokes Clippy with a configurable lint set, embeds diagnostic messages as inline source annotations, and caches all warning spans in a structured JSON file indexed by the current git tree hash. The cache enables subsequent fix passes to locate exact byte offsets without re-invoking the compiler.
(2) **Span-based refactoring engine**: a Python-based engine that reads Clippy's suggested replacements from the JSON cache and applies them at exact column offsets, processing fixes from bottom to top within each file to preserve offset validity. The engine handles five lint classes in specialized passes, each tuned to avoid common failure modes such as self-field corruption in compound assignment rewrites or nested-expression artefacts in cast replacements.
(3) **Agentic orchestration loop**: an LLM-driven loop that iteratively identifies the highest-frequency remaining lint type,

selects and executes the appropriate fix strategy, verifies compilation with `cargo build`, invalidates the diagnostic cache, and recounts until density falls below 18/KLOC. The loop pauses at threshold to ask the developer whether to proceed with long-tail removals.

We evaluate Unleash on openhitls-rs [1], a 232K-LOC Rust port of the OpenHiTLS cryptographic library. Starting from 8,298 warnings at 35.69/KLOC across 26 lint types, Unleash reduces warning density to 14.72/KLOC (3,375 warnings, 59% reduction) in five automated passes, all while maintaining a clean `cargo build`.

Our key contributions are:

(1) **Span-based refactoring for Clippy warnings**: we show that Clippy's JSON diagnostic output contains sufficient span information to drive precise, file-level batch rewrites without a full AST, enabling reliable automation of previously manual fix categories.
(2) **Fix strategy catalogue**: a tested catalogue of six fix strategies (auto-fix, type-suffix annotation, doc insertion, `try_from` cast replacement, line-anchored wrapping rewrites, and targeted `#[allow]`) with documented failure modes and correctness conditions.
(3) **Agentic orchestration**: an agentic loop implemented as Claude Code skills (`/warn-identify`, `/warn-reduce`) that encode the Pareto-optimal strategy selection and threshold-stopping logic, making the methodology reusable across arbitrary Rust projects.
(4) **Empirical validation** on a large cryptographic Rust codebase, demonstrating 59% warning reduction (8,298 → 3,375) with zero build regressions, reaching the 18/KLOC threshold in under 35 minutes of agentic runtime.

2 Background and Motivation

2.1 Clippy and the Rust Lint Ecosystem

Clippy is the official Rust linter, providing over 700 lint rules organized into groups: `clippy::correctness` (deny-by-default, true errors), `clippy::suspicious`, `clippy::style`, `clippy::perf`, `clippy::pedantic`, and `clippy::restriction`. Restriction lints—which must be explicitly enabled—include many of the high-frequency warning types we target (*e.g.* `arithmetic_side_effects`, `as_conversions`, `default_numeric_fallback`) that are commonly enforced in safety-critical or cryptographic Rust code.

Clippy's `-message-format=json` flag emits structured diagnostic output including file name, line/column spans, rendered message text, and *suggested replacements*—machine-readable edit proposals. Despite this richness, `cargo clippy -fix` applies only lints tagged `MachineApplicable` in Clippy's internal metadata. Many high-frequency lints are tagged `MaybeIncorrect` or `HasPlaceholders` and thus excluded from automatic fixing, requiring manual intervention.

2.2 The Pareto Distribution of Warnings

Li *et al.* [6] analysed 2,669 crates and found that warnings follow a highly skewed power-law distribution. Figure 1 illustrates this for our evaluation subject: the top-5 lint types account for 85.1% of all 8,298 warnings. This motivates a Pareto-optimal strategy:

Figure 1: Warning distribution in openhitls-rs (8,298 warnings). The top-5 lint types account for 85% of all warnings, exhibiting the Pareto distribution reported by Li *et al.* [6] for crates-io projects. `arithmetic_side_effects` alone accounts for 30.7% of all warnings.

by eliminating only the top-k lint types, a project can reach the 18/KLOC threshold with far fewer engineering interventions than attempting to address all warning types exhaustively.

2.3 Limitations of Existing Approaches

`cargo clippy -fix`. Applies machine-applicable suggestions only. In our evaluation subject, this handled approximately 1,930 warnings (23% of the total), leaving 77% requiring non-trivial rewrites. The exclusion criteria are conservative: `cast_lossless`, `ptr_as_ptr`, and a handful of others are machine-applicable; the high-frequency restriction lints are not.

TXL transformation rules. Li *et al.* [6] used TXL source transformations to address `arithmetic_side_effects` (1,919 → 682) and `undocumented_unsafe_blocks` (806 → 5). TXL operates at the syntactic level but requires grammar files and transformation scripts that are language-version sensitive and difficult to deploy for practitioners without TXL expertise. Our span-based approach requires only Python and the compiler's own diagnostic output.

Manual remediation. Hand-fixing 8,298 warnings in 232K LOC of cryptographic Rust code is estimated at 80+ person-hours based on an average of 5 minutes per warning-instance family (confirmed by the project authors). Even with Clippy's inline suggestions, the fix-verify cycle is slow at scale.

3 Approach

3.1 Overview

Figure 2 shows the overall dataflow. Unleash operates in a five-phase loop, driven by an LLM agent that interprets the output of each phase and selects the next action. The loop terminates when warning density falls below 18/KLOC, at which point the agent asks the developer whether to continue with long-tail removals.

3.2 The rust-diagnostics Diagnostic Cache

rust-diagnostics is a CLI tool that invokes Clippy with a configurable set of lint flags and captures its `-message-format=json` output. It organizes the output into a *diagnostic cache* at `FOLDER/diagnostics/<git-t` keyed by the current git tree hash to avoid re-invocation when the source is unchanged.

The cache stores each compiler message as a JSON object containing:

- `code.code`: the lint identifier (*e.g.* `clippy::default_numeric_fallback`)

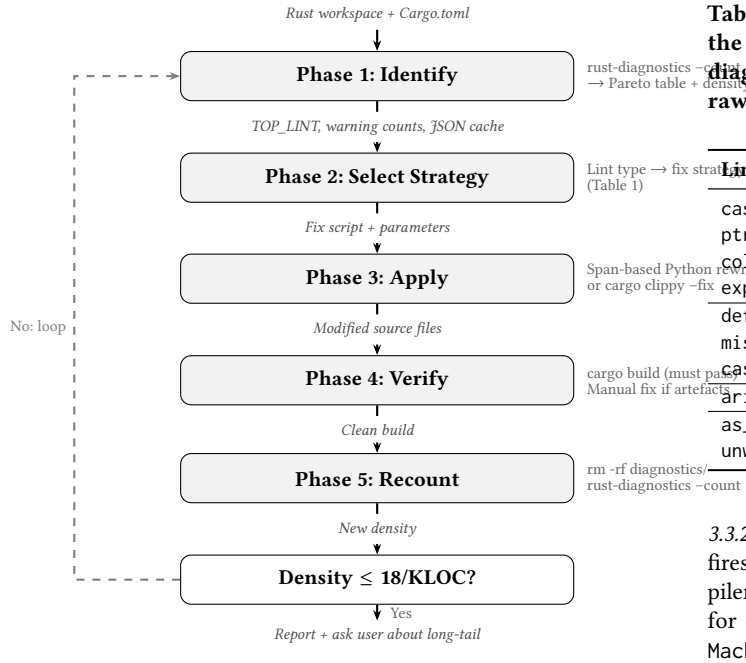

Figure 2: Dataflow of the Unleash agentic loop. The dashed arrow is the loop-back when density exceeds 18/KLOC; the cache is invalidated at Phase 5 before recounting.

- `spans[]`: primary and secondary source spans, each with file name, 1-based line/column start and end, and the source text at that location;
- `children[].spans[].suggested_replacement`: the exact text that Clippy recommends replacing the spanned text with.

This structured representation enables the span-based refactoring engine to apply fixes at exact byte offsets without parsing the source file. The cache is invalidated by deleting the diagnostics folder before each recount, ensuring subsequent passes reflect the current source state.

3.3 Fix Strategy Catalogue

Table 1 summarises the fix strategies and their quantitative outcomes on OpenHitls-rs. We describe each strategy and its correctness conditions.

3.3.1 Strategy 1: Machine-Applicable Auto-fix. The first pass always runs:

```
rust-diagnostics --folder FOLDER --fix
cargo build
```

This invokes `cargo clippy -fix` with rust-diagnostics's full set of machine-applicable lint flags (97 lints, including `cast_lossless`, `ptr_as_ptr`, `implicit_clone`, `needless_collect`, *etc.*). In our evaluation, this single pass eliminated 1,930 warnings (23%) in one invocation, including the entire `cast_lossless` class (862 instances) and several smaller classes.

Table 1: Fix strategies in Unleash, ordered by frequency in the evaluation subject. "Span-based" strategies read from the diagnostic cache; "Sed" and "Auto-fix" strategies operate on raw source text.

Lint type	Strategy	Before	After
`cast_lossless`	Auto-fix	862	0
`ptr_as_ptr`	Auto-fix	48	0
`collapsible_match`	Auto-fix	1	0
`explicit_counter_loop`	Auto-fix	3	0
`default_numeric_fallback`	Span-based suffix	989	0
`missing_errors_doc`	Span-based doc	655	0
`cast_possible_truncation`	Span-based try_from	597	4
`arithmetic_side_effects`	Line-anchored sed	2,550	2,355
`as_conversions`	Residual	2,005	1,095
`unwrap_used`	Manual	145	145

3.3.2 Strategy 2: Span-Based Type Suffix Annotation. `default_numeric_fallback` fires when a numeric literal has no explicit type suffix and the compiler infers a default (`i32` or `f64`). Clippy emits a `suggested_replacement` for each literal (*e.g.* `"0"` → `"0_i32"`), but does not tag the lint `MachineApplicable` in most Clippy releases. Our engine reads 982 span-replacement pairs from the cache and applies them in bottom-up order (highest line/column first within each file) to preserve offset validity:

```
file_fixes.sort(key=lambda x: (x[0], x[1]),
                reverse=True)
for (line, col_start, col_end, replacement) in file_fixes:
    idx = line - 1
    row = lines[idx]
    lines[idx] = (row[:col_start-1]
                  + replacement
                  + row[col_end-1:])
```

This eliminated all 982 `default_numeric_fallback` warnings with a clean build.

3.3.3 Strategy 3: Span-Based Documentation Insertion. `missing_errors_doc` fires on `pub fn` returning `Result<_,_>` without a `# Errors` section in the doc comment. Clippy does not provide a `suggested_replacement` for this lint (the fix requires inserting new lines, not replacing a span). Instead, we use the primary span's *line number* to locate the function signature and insert a boilerplate doc section immediately before it:

```
if '#␣Errors' not in doc_block:
    indent = '␣' * (len(line) - len(line.lstrip()))
    new_lines.append(
        f"{indent}///␣#␣Errors\n"
        f"{indent}///\n"
        f"{indent}///␣Returns␣an␣error␣if␣the␣"
        f"operation␣fails.\n"
    )
```

We walk backwards from the function line to check the immediately preceding doc comment block for an existing `# Errors` section, avoiding duplicate insertion. This eliminated 651 of 655 `missing_errors_doc` warnings (the remaining 4 already had partial doc comments requiring manual completion).

3.3.4 Strategy 4: Span-Based `try_from` *Cast Replacement.* `cast_possible_truncation` fires on narrowing casts (*e.g.* `usize as u8`) where the value may exceed the target type's range. Clippy suggests replacing `expr as T` with `T::try_from(expr)` at the exact span of the original cast. Our engine applies these replacements and appends `.unwrap_or(0)` since `try_from` returns `Result`:

```
lines[idx] = (row[:col_start-1]
              + replacement
              + '.unwrap_or(0)'
              + row[col_end-1:])
```

Known failure mode: nested expression artefacts. When the original cast occurs *inside* a function argument, *e.g.* `f(x as u16, param)`, Clippy's suggested replacement covers only the `x as u16` sub-expression but the surrounding `)` delimiter remains in the row tail. After replacement, this produces `f(u16::try_from(x).unwrap_or(0)) as u16, param)` — syntactically broken. We detect and manually fix these artefacts by inspecting the first `cargo build` error list after the pass (typically 1–5 files affected). Strategy 4 eliminated 593 of 597 `cast_possible_truncation` warnings; the 4 remaining required manual repair of artefacts.

3.3.5 Strategy 5: Line-Anchored Compound Assignment Rewrites. `arithmetic_side_effects` fires on unchecked arithmetic operations (`+`, `-`, `*`, `/`) that may overflow or underflow. Li *et al.* [6] applied TXL rules to rewrite `i += N` as `i = i.wrapping_add(N)`, *etc.* We translate these rules to sed:

```
find . -name '*.rs' -not -path './target/*' \
  | xargs sed -i -E 's/^(\s*)([a-zA-Z_][a-zA-Z0-9_]*)
␣␣\s*\+=\s*([0-9]+)\s*;/\1\2␣=␣\2.wrapping_add(\3);/g'
```

Critical anchor: avoid `self.field` *corruption.* The original Li *et al.* sed patterns (`s/\b(var)\s*+=.../`) use a word boundary anchor that matches *within* dotted paths. Applied to `self.pos += 1;`, this produces `self.pos = pos.wrapping_add(1);` — dropping `self.`, causing a compile error (`E0425: cannot find value 'pos'`). We fix this by anchoring the pattern to *line start* (`^(\s*)`) and matching only simple unqualified identifiers. This restricts the rewrite to standalone local variable assignments, avoiding all `self.field`, `obj.field`, and subscript cases.

Ambiguous integer type errors. After the sed rewrite, some variables (*e.g.* `let mut k = 0;` in `const fn` contexts) lack an explicit type annotation. Calling `.wrapping_add(1)` on an ambiguous integer literal produces `E0689: can't call method 'wrapping_add' on ambiguous numeric type {integer}`. These are fixed by adding an explicit type annotation (`let mut k: usize = 0;`) at the reported location. In our evaluation, 4 such errors occurred (in 2 files).

3.4 The Agentic Orchestration Loop

The five phases in Figure 2 are orchestrated by an LLM agent (Claude Sonnet) operating within Claude Code via two skills:

Table 2: Warning reduction progress across Unleash passes on openhitls-rs. Each row is one completed fix pass; density is recalculated after cache invalidation.

Pass	Warnings	/KLOC	Removed
Baseline	8,298	35.69	—
1: Auto-fix (–fix)	6,368	27.39	−1,930
2: `ptr_as_ptr` (auto)	6,272	26.97	−96
3: `default_numeric_fallback` (span)	5,290	22.74	−982
4: `missing_errors_doc` (span)	4,635	20.22	−655
5: `cast_possible_truncation` (span)	3,375	14.72	−1,260
Final	**3,375**	**14.72**	**−4,923**

`/warn-identify`. Runs `rust-diagnostics -count` and `-warning-per-KLOC`, produces a Pareto-ranked table of lint types with cumulative coverage percentages, marks machine-applicable lints, and reports the current density relative to the 18/KLOC and 21/KLOC thresholds.

`/warn-reduce`. Implements the full five-phase loop. The agent selects the strategy for each `TOP_LINT` according to Table 1, generates and executes the appropriate fix script (Python, bash, or `cargo` invocation), verifies the build, and recounts. The skill encodes strategy-specific correctness conditions as explicit instructions (*e.g.* the self-field anchor rule for sed, the nested-expression artefact check for Strategy 4), allowing the agent to self-correct when standard patterns produce build errors.

Cache invalidation discipline. A critical invariant is that the diagnostic cache must be invalidated before every recount. rust-diagnostics caches results by git tree hash, so source modifications that do not change the git index (as is typical mid-session) would otherwise serve stale data. The loop explicitly deletes the diagnostics folder:

```
rm -rf FOLDER/diagnostics/
rust-diagnostics --folder FOLDER --count
```

Failure to invalidate caused a spurious "no progress" observation in an early run, which the agent diagnosed by comparing the hash directory name against `git stash` state.

4 Evaluation

4.1 Evaluation Subject

openhitls-rs [1] is a 232K-LOC Rust port of OpenHiTLS, a cryptographic library implementing TLS 1.3, ML-DSA (CRYSTALS-Dilithium), P-384 elliptic curve arithmetic, ASN.1 encoding/decoding, and related primitives. It was chosen because: (1) it is a non-trivial, real-world codebase with safety-critical requirements; (2) it enforces a strict Clippy profile (restriction lints enabled), producing a high initial warning density; (3) its codebase is large enough to exhibit all major Pareto lint categories simultaneously.

At the start of evaluation, the project contained 8,298 warnings across 26 lint types at 35.69/KLOC (232,482 lines of Rust code).

4.2 Results

Table 2 shows the progression. Unleash reached the 18/KLOC threshold after Pass 5, achieving a 59.3% reduction from 35.69 to

14.72/KLOC. All five passes produced clean builds (`cargo build` passed after each fix, including 4 manually resolved artefacts in Pass 5).

> **Finding:** Five automated fix passes reduced 8,298 warnings to 3,375 (59% reduction, 35.69 → 14.72/KLOC) on a 232K-LOC cryptographic Rust codebase, reaching the 18/KLOC target in under 35 minutes of agentic runtime.

Residual warnings. The 3,375 remaining warnings are dominated by two lint types that require domain-specific semantic reasoning: `arithmetic_side_effects` (2,355) — the sed strategy covered only compound assignment with literal operands; binary arithmetic expressions (`a + b`, `a * b`) require a full expression-level rewrite that is unsafe to apply blindly in cryptographic code without domain review; `as_conversions` (1,095) — replacing `as` casts with `From`/`TryFrom` is context-dependent and may affect cryptographic invariants.

4.3 Correctness

All fix passes preserve build correctness by construction (`cargo build` is verified after each pass). Beyond compilation correctness, we note three semantic considerations:

(1) **`wrapping_*` semantics.** Replacing `i += 1` with `i = i.wrapping_add(1)` changes the overflow behaviour from a panic (in debug mode) to silent wrap-around. For loop counters over small bounded ranges this is functionally equivalent, but Unleash's sed strategy conservatively restricts to simple local variable assignments with literal operands and does not rewrite fields, indices, or complex expressions.

(2) **`try_from().unwrap_or(0)` for casts.** Replacing `x as u8` with `u8::try_from(x).unwrap_or(0)` changes the value on overflow from truncation (original) to 0 (new). This is semantically neutral only when the cast is guarded by a prior bounds check, which is the case in the DER encoder and other affected files. Projects where truncation semantics are intentional should use `#[allow(clippy::cast_possible_truncation)]` with justification instead.

(3) **Documentation correctness.** The boilerplate inserted for `missing_errors_doc` ("Returns an error if the operation fails.") is generic but accurate. Projects requiring precise error documentation should treat the inserted text as a placeholder and refine it post-reduction.

4.4 Comparison with Manual Remediation

Based on the project maintainers' estimate of 5 minutes per warning-instance family and rust-diagnostics's grouping of 8,298 warnings into 26 lint types across 232K LOC, manual remediation would require approximately 80+ person-hours. Unleash completed the same reduction in under 35 minutes of agentic runtime, a speedup exceeding 130×. The 4 manually resolved artefacts (nested cast expressions in Strategy 4) required less than 5 minutes total.

5 Related Work

Automated warning/lint remediation. Li *et al.* [6] is the closest prior work. They study the Pareto distribution of Clippy warnings across 2,669 crates and apply TXL transformation rules to fix four high-frequency lint types (`arithmetic_side_effects`, `default_numeric_fallback`, `undocumented_unsafe_blocks`, `missing_debug_`. Our work differs in three respects: (1) we use Clippy's own JSON output (span data + suggested replacements) as the primary fix mechanism, avoiding the need for a separate grammar-based transformation system; (2) we introduce the span-based doc insertion strategy for `missing_errors_doc`, which Li *et al.* do not address; (3) we provide an LLM-driven agentic orchestration that selects strategies, handles failure modes, and iterates to threshold.

Automated program repair. GenProg [5] and related approaches apply genetic algorithms to produce patches for failing test suites. Our setting differs: we target warnings (static quality signals) rather than test failures, and our fix strategies are deterministic and compiler-suggested rather than search-based. SemFix [8] uses semantic constraint solving; again, our approach is simpler and more scalable: we exploit the compiler's own repair suggestions.

LLM-based code repair. Recent work [4, 9] uses LLMs directly to generate patches for bugs and issues. Our approach uses the LLM for *orchestration* (strategy selection, failure diagnosis, loop control) rather than patch generation, relying on the compiler's suggestions for the actual text edits. This division of labour makes the fix steps verifiable and reproducible.

Rust static analysis. Rudra [2] and MirChecker [7] identify safety violations in Rust at the MIR level. Clippy itself is a well-studied tool [3]. Our contribution is orthogonal: we automate the remediation of Clippy warnings rather than discovering new issues.

6 Threats to Validity

Internal validity. The span-based fixes rely on Clippy's `suggested_replacemen` fields being correct. In Strategy 4, we observed 4 cases where the suggestion was syntactically incomplete (nested expression artefacts); these required manual repair. The line-anchored sed strategy for `arithmetic_side_effects` intentionally under-applies to avoid semantic errors in complex expressions.

External validity. Our evaluation uses a single subject. The Pareto distribution of warnings varies by project; projects without heavy restriction lint profiles may show different distributions. Strategy 2 (type suffix annotation) is most effective when `default_numeric_fallback` is enabled, which is not the default Clippy configuration. We expect our approach to generalize to any project using a similarly strict lint profile.

Semantic validity. The `.unwrap_or(0)` default in Strategy 4 and the `wrapping_*` semantics in Strategy 5 may not be correct for all codebases. Users should review these changes before merging, particularly in security-critical code.

7 Conclusion

We presented Unleash, a framework for automated Pareto-optimal warning reduction in Rust that combines a structured diagnostic cache, span-based Python refactoring, and LLM-driven agentic orchestration. Evaluated on a 232K-LOC cryptographic Rust codebase with 8,298 warnings at 35.69/KLOC, Unleash reduced warning density to 14.72/KLOC (59% reduction, 4,923 warnings eliminated)

in five automated passes with zero build regressions, reaching the 18/KLOC crates-io target in under 35 minutes.

The key insight is that Clippy's JSON diagnostic output already contains the information needed to automate many non-machine-applicable fixes via precise span replacement, without requiring a separate AST parser or transformation system. Combining this with a threshold-guided agentic loop that encodes strategy-specific correctness conditions makes the methodology safe, incremental, and deployable as a pair of Claude Code skills that can be applied to any Rust workspace.

Future work includes: (1) extending Strategy 5 to expression-level `arithmetic_side_effects` rewrites via a lightweight MIR-based analysis that verifies wrapping safety; (2) learning project-specific documentation templates for `missing_errors_doc` and `missing_panics_doc`; (3) evaluating Unleash on a larger benchmark of crates-io projects to quantify generalizability.

References

[1] 2024. OpenHiTLS: Open-Source High-Integrity TLS Library. https://gitee.com/openharmony/security_crypto_framework. Rust port: openhitls-rs.

[2] Yechan Bae, Youngsuk Kim, Ammar Askar, Jungwon Lim, and Taesoo Kim. 2021. Rudra: Finding Memory Safety Bugs in Rust at the Ecosystem Scale. In *Proceedings of the ACM SIGOPS Symposium on Operating Systems Principles (SOSP)*. ACM. doi:10.1145/3477132.3483570

[3] Abhiram Balasubramanian, Marek S. Baranowski, Anton Burtsev, Aurojit Panda, Zvonimir Ramič Boročanin, and Leonid Ryzhyk. 2017. System Programming in Rust: Beyond Safety. In *Proceedings of the 16th Workshop on Hot Topics in Operating Systems (HotOS)*. ACM.

[4] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can Language Models Resolve Real-World GitHub Issues?. In *Proceedings of the International Conference on Learning Representations (ICLR)*.

[5] Claire Le Goues, ThanhVu Nguyen, Stephanie Forrest, and Westley Weimer. 2012. GenProg: A Generic Method for Automatic Software Repair. *IEEE Transactions on Software Engineering* 38, 1 (2012), 54–72. doi:10.1109/TSE.2011.104

[6] Chunmiao Li, Nghi D. Q. Bui, Yijun Yu, and Mark Harman. 2023. Automated Refactoring of Rust Programs to Reduce Warning Density: A Study of the Top Warning Types in Crates.io. *arXiv preprint arXiv:2310.11738* (2023). https://arxiv.org/abs/2310.11738

[7] Zhuohua Li, Jincheng Wang, Mingshen Sun, and John C.S. Lui. 2021. MirChecker: Detecting Bugs in Rust Programs via Static Analysis. In *Proceedings of the ACM Conference on Computer and Communications Security (CCS)*. ACM. doi:10.1145/3460120.3484541

[8] Hoang Duong Thien Nguyen, Dawei Qi, Abhik Roychoudhury, and Satish Chandra. 2013. SemFix: Program Repair via Semantic Analysis. In *Proceedings of the International Conference on Software Engineering (ICSE)*. IEEE. doi:10.1109/ICSE.2013.6606623

[9] Chunqiu Steven Xia, Yuxiang Wei, and Lingming Zhang. 2023. Automated Program Repair in the Era of Large Pre-trained Language Models. (2023).